

PyTorch is an open-source deep learning library, originally developed by Meta Platforms and currently developed with support from the Linux Foundation. The successor to Torch, PyTorch provides a high-level API that builds upon optimised, low-level implementations of deep learning algorithms and architectures, such as the Transformer, or SGD. Notably, this API simplifies model training and inference to a few lines of code. PyTorch allows for automatic parallelization of training and, internally, implements CUDA bindings that speed training further by leveraging GPU resources.

PyTorch utilises the tensor as a fundamental data type, similarly to NumPy. Training is facilitated by a reversed automatic differentiation system, Autograd, that constructs a directed acyclic graph of the operations (and their arguments) executed by a model during its forward pass. With a loss, backpropagation is then undertaken.

As of 2025, PyTorch remains one of the most popular deep learning libraries, alongside others such as TensorFlow and Keras. It can be installed using Anaconda package managers. A number of commercial deep learning systems are built on top of PyTorch, including ChatGPT, Tesla Autopilot, Uber's Pyro, and Hugging Face's Transformers.

History

In 2001, Torch was written and released under a GPL by the Idiap Research Institute. It was a machine-learning library written in C++ and CUDA, supporting methods including neural networks, support vector machines (SVM), hidden Markov models, etc. Around 2010, it was rewritten by Ronan Collobert, Clement Farabet and Koray Kavuckuoglu. This was known as Torch7 or LuaTorch. This was written so that the backend was in C and the frontend was in Lua. In mid-2016, some developers refactored it to decouple the frontend and the backend, with strong influence from torch-autograd and Chainer. In turn, torch-autograd was influenced by HIPS/autograd. Development on Torch7 ceased in 2018 and was subsumed by the PyTorch project.

Meta (formerly known as Facebook) operates both PyTorch and Convolutional Architecture for Fast Feature Embedding (Caffe2), but models defined by the two frameworks were mutually incompatible. The Open Neural Network Exchange (ONNX) project was created by Meta and Microsoft in September 2017 to decouple deep learning frameworks from hardware-specific runtimes, allowing models to be converted between frameworks and optimized for execution providers like NVIDIA's TensorRT. Caffe2 was merged into PyTorch at the end of March 2018. In September 2022, Meta announced that PyTorch would be governed by the independent PyTorch Foundation, a newly created subsidiary of the Linux Foundation.

PyTorch 2.0 was released on 15 March 2023, introducing TorchDynamo, a Python-level compiler that makes code run up to two times faster, along with significant improvements in training and inference performance across major cloud platforms.

PyTorch tensors

PyTorch defines a class called Tensor (`torch.Tensor`) to store and operate on homogeneous multidimensional rectangular arrays of numbers. PyTorch supports various sub-types of multi-dimensional arrays, or Tensors. PyTorch Tensors are similar to NumPy Arrays, but can also be operated on by a CUDA-capable NVIDIA GPU. PyTorch has also been developing support for other GPU platforms, for example, AMD's ROCm and Apple's Metal Framework.

PyTorch neural networks

PyTorch defines a module called nn (`torch.nn`) to describe neural networks and to support training. This module offers a comprehensive collection of building blocks for neural networks, including various layers and activation functions, enabling the construction of complex models. Networks are built by inheriting from the `torch.nn` module and defining the sequence of operations in the `forward()` function.

PyTorch Serialized File Format

Pytorch can save and load models using its own file format, which is a ZIP64 archive containing the model weights in a Python pickle file, and other information such as the byte order. The file extensions .pt and .pth are commonly used for these files.

Example

The following program shows the low-level functionality of the library with a simple example.

The following code block defines a neural network with linear layers using the nn module.

See also

[Comparison of deep learning software](#)

[Comparison of machine learning software](#)

[Differentiable programming](#)

[DeepSpeed](#)

[Open-source artificial intelligence](#)

[PyTorch Lightning](#)

References

External links

Official website